

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	G. Jaswanth	Reg. No.:	192472235
Section / Batch:	2024	Date:	

Code of Conduct

I G. Jaswanth - 192472235 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G. Jaswanth

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach

Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	4	4	4	3	3	18	92
2	4	4	4	3	3	18	
3	4	4	4	3	4	19	
4	4	4	3	4	4	19	
5	4	3	3	4	4	18	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Reference Resolution Using Multiple Constraints

Given Discourse

“Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it.”

1. Problem Understanding

The objective is to identify the antecedent of each referring expression by applying **gender, number, recency, and semantic compatibility constraints**.

The referring expressions are:

- **she** → Meena / Kavya
- **one** → a laptop
- **her** → Meena / Kavya
- **She** → Meena / Kavya
- **her** → Meena / Kavya
- **it** → laptop

The main challenge is that **Meena and Kavya are both female and singular**, so gender and number agreement alone cannot resolve all pronouns. The algorithm must therefore combine multiple constraints.

2. Entity and Referring Expression Identification

Entity / Expression	Type	Possible Reference
Meena	Person	—
Kavya	Person	—
a laptop	Object	—
she	Pronoun	Meena / Kavya
one	Pronoun	laptop
her	Possessive pronoun	Meena / Kavya
She	Pronoun	Meena / Kavya
her	Object pronoun	Meena / Kavya
it	Pronoun	laptop

3. Possible Antecedents

Referring Expression	Possible Antecedents
she	Meena, Kavya
one	laptop
her (her project)	Meena, Kavya
She (She thanked)	Meena, Kavya

Referring Expression	Possible Antecedents
her (thanked her)	Meena, Kavya
it	laptop

4. Constraint Analysis

Gender and Number Agreement

Both **Meena** and **Kavya** are singular female names.

Therefore:

- Meena → **she** ✓
- Kavya → **she** ✓
- Meena → **her** ✓
- Kavya → **her** ✓

Thus, gender and number **cannot independently resolve the ambiguity**.

Recency Constraint

Kavya is the most recently mentioned person before “**she**”.

Therefore:

she → **Kavya** is initially favored by recency.

However, recency alone is not sufficient because semantic and discourse constraints must also be considered.

Semantic Compatibility

The sentence says:

“Meena gave Kavya a laptop because she needed one for her project.”

The person who **needs a laptop for a project** is naturally interpreted as **Kavya**, the recipient of the laptop.

Therefore:

she → **Kavya**

and:

her project → **Kavya's project**

Discourse Coherence

The most coherent interpretation is:

Meena gave Kavya a laptop → Kavya needed the laptop → Kavya thanked Meena → Kavya started using the laptop.

This produces a consistent sequence of actions.

5. Reference Resolution Algorithm

Pseudocode

ALGORITHM ReferenceResolution(Text)

INPUT:

Text containing entities and referring expressions

OUTPUT:

Resolved antecedents and resolved discourse

BEGIN

1. Identify all named entities in Text

Entities $\leftarrow$ ExtractEntities(Text)

2. Identify all referring expressions

References $\leftarrow$ ExtractPronouns(Text)

3. FOR each reference R in References DO

Candidates $\leftarrow$ FindPreviousEntities(R, Entities)

FOR each candidate C in Candidates DO

Score[C] $\leftarrow$ 0

// Gender constraint

IF Gender(C) matches Gender(R) THEN

Score[C] $\leftarrow$ Score[C] + 1

ELSE

Score[C] $\leftarrow$ Score[C] - 1

END IF

// Number constraint

IF Number(C) matches Number(R) THEN

Score[C] $\leftarrow$ Score[C] + 1

ELSE

Score[C] $\leftarrow$ Score[C] - 1

END IF

// Recency constraint

IF C is recently mentioned THEN

Score[C] $\leftarrow$ Score[C] + 1

END IF

// Semantic compatibility

IF C is semantically compatible with R THEN

Score[C] $\leftarrow$ Score[C] + 2

ELSE

Score[C] $\leftarrow$ Score[C] - 1

END IF

// Discourse coherence

IF C produces a coherent interpretation THEN

Score[C] $\leftarrow$ Score[C] + 2

END IF

END FOR

Select candidate C having the highest Score

Assign $R \rightarrow C$

END FOR

4. Replace referring expressions with their selected antecedents.

5. Return resolved discourse.

END

6. Applying the Algorithm

Reference 1: “**she**”

Candidates:

- Meena
- Kavya

Constraint	Meena	Kavya
Gender	✓	✓
Number	✓	✓
Recency	✗	✓
Semantic compatibility	Possible	✓
Discourse coherence	Moderate	✓
Overall	Lower	Higher

Resolution:

she → **Kavya**

Reference 2: “**one**”

“One” refers to something that can be needed for a project.

one → **laptop**

Reference 3: “**her project**”

The antecedent must be a female person who owns a project.

Based on the previous resolution:

her → **Kavya**

Therefore:

her project → **Kavya's project**

Reference 4: “**She thanked her**”

The previous sentence establishes Kavya as the person who needed the laptop.

Therefore:

She → Kavya

For the object “**her**”, the natural complementary interpretation is:

her → Meena

Thus:

Kavya thanked Meena.

Reference 5: “**it**”

The only suitable object is the laptop.

it → laptop

7. Final Coreference Chains

Person Chain 1

Kavya → she → her (project) → She

Person Chain 2

Meena → her (thanked her)

Object Chain

laptop → one → it

8. Resolved Discourse

**“Meena gave Kavya a laptop because Kavya needed a laptop for Kavya's project.
Kavya thanked Meena and started using the laptop.”**

This version removes the ambiguity and makes all entity relationships explicit.

9. How the Algorithm Avoids Incorrect Resolution

When multiple antecedents have similar grammatical properties, relying only on **gender and number** can produce incorrect results. The proposed algorithm therefore combines multiple constraints.

For example, both Meena and Kavya satisfy:

female + singular → “she”

The algorithm then uses **recency, semantic compatibility, grammatical context, and discourse coherence** to select the most appropriate antecedent.

Therefore, instead of selecting an antecedent based on a single rule, the algorithm uses a **scoring-based constraint approach**. This improves the reliability of reference resolution when several candidates have similar grammatical properties.

Conclusion

The algorithm successfully resolves the ambiguous references by combining **gender, number, recency, semantic compatibility, and discourse coherence**. The final interpretation is that **Kavya needed the laptop, Kavya thanked Meena, and Kavya used the laptop**, producing a coherent discourse.

Q2. Algorithm for Entity Chains and Discourse Coherence

Given Text

“Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application.”

1. Problem Understanding

The objective is to identify **entities and referring expressions**, link expressions that refer to the same entity, construct **entity chains**, and determine whether the discourse is coherent.

The main entity in the given text is the **laptop**, while **Anjali** is the main person entity.

The referring expressions are:

- **The laptop**
- **She**
- **it**
- **Anjali**
- **the computer**

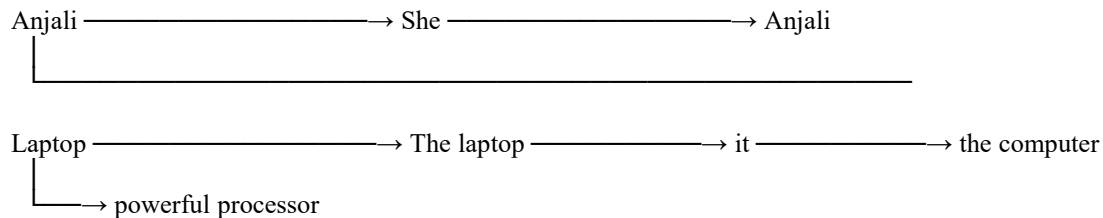
The algorithm determines whether these expressions maintain continuity across the sentences.

2. Identify Entities and Referring Expressions

Sentence	Entity / Expression	Type	Refers To
S1	Anjali	Person	Anjali
S1	a new laptop	Object	Laptop
S2	The laptop	Object	Same laptop
S2	a powerful processor	Object	Processor
S3	She	Pronoun	Anjali
S3	Python	Software	Python
S3	it	Pronoun	Laptop
S4	Anjali	Person	Anjali
S4	the computer	Object	Laptop
S4	machine learning application	Object	Application

3. Entity Linking

The referring expressions are linked according to their meaning and context.



Entity Resolution

- **She** → **Anjali**
- **The laptop** → **new laptop**
- **it** → **laptop**
- **the computer** → **laptop**

The context supports these links because Anjali is the person performing the actions, while the laptop is the object being used.

4. Entity Chains

Entity Chain 1 – Anjali

Anjali → She → Anjali

This chain appears across **S1, S3, and S4**.

Entity Chain 2 – Laptop

new laptop → The laptop → it → the computer

This chain appears across **S1, S2, S3, and S4**.

Entity Chain 3 – Processor

powerful processor

The processor is introduced in S2 but is not subsequently referred to.

Entity Chain 4 – Application

machine learning application

The application is introduced in S4.

5. Continuity Analysis

Entity continuity can be estimated by checking whether important entities are maintained across consecutive sentences.

Sentence	Main Entities	Entity Continuity
----------	---------------	-------------------

Sentence	Main Entities	Entity Continuity
S1	Anjali, Laptop	Initial entities introduced
S2	Laptop, Processor	Laptop continues
S3	Anjali, Laptop, Python	Both major entities continue
S4	Anjali, Laptop, Application	Major entities continue

The two important entities, **Anjali and the laptop**, remain connected throughout the discourse.

Therefore, the text has **high entity continuity**.

6. Coherence Decision

Result: COHERENT

The discourse is coherent because:

- **Anjali** remains the main person/entity throughout the text.
- The **laptop** remains the main object across multiple sentences.
- Pronouns such as “**She**” and “**it**” have clear antecedents.
- “**The computer**” is semantically compatible with the previously mentioned laptop.
- Each sentence logically develops the previous sentence.

Logical Flow

Anjali buys laptop



Laptop has processor



Anjali installs Python on laptop



Anjali uses laptop/computer



Develops machine learning application

This creates a continuous and logically connected discourse.

7. Pseudocode

ALGORITHM CoherenceAnalysis(Text)

INPUT:

A discourse containing multiple sentences

OUTPUT:

Entity chains and coherence decision

BEGIN

1. Divide Text into individual sentences.
Sentences $\leftarrow$ SentenceSegmentation(Text)
2. Identify all entities and referring expressions.
Entities $\leftarrow$ ExtractEntities(Text)
References $\leftarrow$ ExtractReferringExpressions(Text)
3. FOR each referring expression R DO

Candidates $\leftarrow$ FindPreviousEntities(R)

FOR each candidate C DO

Score[C] $\leftarrow$ 0

IF semantic_type(R) matches semantic_type(C) THEN
Score[C] $\leftarrow$ Score[C] + 2
END IF

IF grammatical_number(R) matches C THEN
Score[C] $\leftarrow$ Score[C] + 1
END IF

IF C is recently mentioned THEN
Score[C] $\leftarrow$ Score[C] + 1
END IF

IF C creates a coherent interpretation THEN
Score[C] $\leftarrow$ Score[C] + 2
END IF

END FOR

Select candidate with highest Score.

Link R to selected candidate.

END FOR

4. Construct entity chains from all linked entities.

5. FOR each consecutive pair of sentences DO

CommonEntities $\leftarrow$ FindSharedEntities(Sentence[i],
Sentence[i+1])

IF CommonEntities is not empty THEN
Increase ContinuityScore
ELSE
Decrease ContinuityScore
END IF

END FOR

6. Calculate or estimate overall entity continuity.

7. IF ContinuityScore is high THEN
Coherence $\leftarrow$ "Coherent"
ELSE
Coherence $\leftarrow$ "Low Coherence"
END IF

8. Display entity chains and coherence result.

END

8. Applying the Algorithm

Reference: "She"

Candidates:

- Anjali

- Laptop
- Processor

The pronoun “**She**” requires a singular female person.

Therefore:

She → Anjali

Reference: “it”

Candidates include:

- Anjali
- Laptop
- Processor
- Python

The verb “**installed Python on it**” requires an object/location compatible with installation.

Therefore:

it → laptop

Reference: “the computer”

The laptop is a computing device, making “**the computer**” semantically compatible with the previously mentioned laptop.

Therefore:

the computer → laptop

9. Final Entity Chains

PERSON:

Anjali → She → Anjali

OBJECT:

new laptop → The laptop → it → the computer

OTHER ENTITIES:

powerful processor

Python

machine learning application

10. What Happens if an Entity Chain Is Broken?

Breaking an entity chain can reduce discourse coherence because the reader or NLP system may no longer understand how consecutive sentences are related.

For example:

“Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. **The refrigerator was expensive.**”

The final sentence introduces an unrelated entity, “**refrigerator**”, breaking the established **Anjali–laptop** entity chains.

This causes:

- Loss of topic continuity.
- Weak connection between sentences.
- Reduced discourse coherence.
- Difficulty identifying the main topic.
- Poorer understanding by an NLP system.

Conclusion

The given discourse is **highly coherent** because the entity chains for **Anjali** and the **laptop** are maintained across the sentences. Entity-chain analysis helps NLP systems measure topic continuity and detect sudden or unrelated shifts that can reduce discourse coherence.

Q3. Algorithm for Identifying Discourse Relations

Given Text

“The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again.”

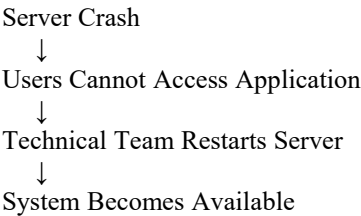
1. Problem Understanding

The objective is to identify the **discourse relations** between consecutive sentences using explicit discourse markers and contextual meaning.

The important relations in this discourse are:

- **Cause–Effect**
- **Problem–Solution**
- **Sequence**

The discourse follows a clear progression:



2. Divide the Discourse into Discourse Units

Unit	Sentence
U1	The server crashed unexpectedly.
U2	As a result, users could not access the application.
U3	The technical team restarted the server.

Unit	Sentence
U4	After that, the system became available again.

3. Identify Explicit Discourse Markers

Discourse Marker	Location	Indicated Relation
As a result	U2	Cause–Effect
After that	U4	Sequence

These markers provide direct evidence about how the discourse units are connected.

4. Semantic Relationship Analysis

U1 → U2: Cause–Effect

The server crashed unexpectedly.

As a result, users could not access the application.

The server crash is the **cause**, while users being unable to access the application is the **effect**.

Relation: Cause–Effect

U2 → U3: Problem–Solution

Users could not access the application.

The technical team restarted the server.

U2 describes the **problem**, while U3 describes the action taken by the technical team to address the problem.

Relation: Problem–Solution

U3 → U4: Sequence

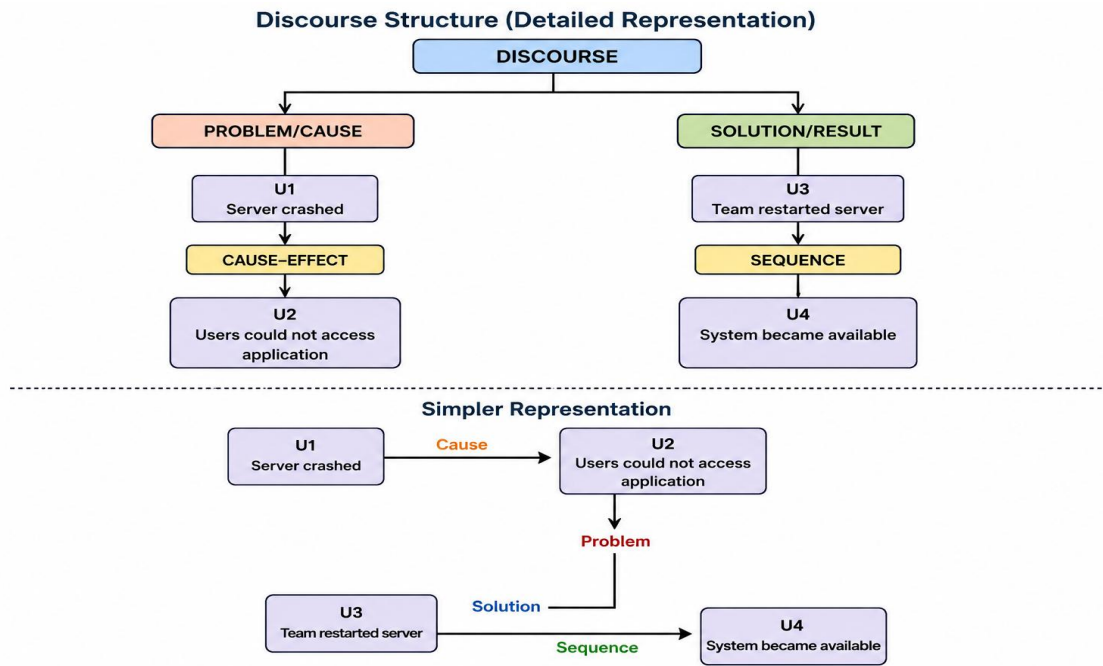
The technical team restarted the server.

After that, the system became available again.

The system becoming available occurs after the server is restarted.

Relation: Sequence

5. Discourse Structure Representation



6. Pseudocode

ALGORITHM DiscourseRelationAnalysis(Text)

INPUT:

A sequence of related sentences

OUTPUT:

Discourse relations and discourse structure

BEGIN

1. Divide Text into discourse units.

Units $\leftarrow$ SentenceSegmentation(Text)

2. FOR each Unit U DO

 Identify explicit discourse markers.

 IF marker = "as a result" THEN

 Mark relation as Cause-Effect

 ELSE IF marker = "after that" THEN

 Mark relation as Sequence

 END IF

END FOR

3. FOR each consecutive pair (U_i, U_{i+1}) DO

 IF U_i describes a cause AND

 U_{i+1} describes its consequence THEN

 Relation $\leftarrow$ Cause-Effect

 ELSE IF U_i describes a problem AND

 U_{i+1} describes an action addressing it THEN

 Relation $\leftarrow$ Problem-Solution

 ELSE IF U_{i+1} occurs after U_i in time THEN

Relation ← Sequence

ELSE IF U_i provides additional information about U_{i-1} THEN
Relation ← Elaboration

ELSE
Relation ← Unknown
END IF

END FOR

4. Construct a discourse structure using:

Units
Relations
Discourse markers

5. Return the identified relations and structure.

END

7. Applying the Algorithm

Units	Evidence	Identified Relation
$U_1 \rightarrow U_2$	“As a result” + crash causes access failure	Cause–Effect
$U_2 \rightarrow U_3$	Access problem followed by server restart	Problem–Solution
$U_3 \rightarrow U_4$	“After that” indicates temporal order	Sequence

Final Result

$U_1 \rightarrow U_2$: Cause–Effect

$U_2 \rightarrow U_3$: Problem–Solution

$U_3 \rightarrow U_4$: Sequence

8. Justification of Coherence

The identified relations make the discourse logically coherent because each sentence contributes to a continuous **problem** → **consequence** → **solution** → **result** structure.

- **U1** introduces the technical problem.
- **U2** explains the consequence of the problem.
- **U3** describes the corrective action.
- **U4** describes the positive outcome of that action.

Therefore, there are no unrelated topic shifts, and each discourse unit is logically connected to the next.

Conclusion

The algorithm successfully identifies **Cause–Effect**, **Problem–Solution**, and **Sequence** relations using both explicit discourse markers and contextual meaning. These relationships establish a clear logical progression and enable an NLP system to understand the overall structure and coherence of the discourse.

Q4. Algorithm for Dialogue Act Classification

Given Conversation

User: “Hello, I need help with my assignment.”

Agent: “Sure, what problem are you facing?”

User: “I do not understand machine learning.”

Agent: “I can explain the basic concepts to you.”

1. Problem Understanding

The objective is to design an algorithm that identifies the **communicative intention** of each utterance and assigns an appropriate **dialogue act**.

The available dialogue acts are:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

The system should analyze each utterance and produce the corresponding dialogue-act sequence.

2. Preprocessing of Utterances

Each utterance is processed to identify:

- Important words and phrases
- Sentence type
- Question indicators
- Request indicators
- Greeting expressions
- Offer expressions
- Informative statements

Preprocessed Conversation

Speaker	Utterance	Important Cue
User	Hello, I need help with my assignment.	“Hello”, “need help”
Agent	Sure, what problem are you facing?	“what”
User	I do not understand machine learning.	“do not understand”
Agent	I can explain the basic concepts to you.	“I can explain”

3. Dialogue Act Identification

Utterance 1

“Hello, I need help with my assignment.”

The word **“Hello”** is a greeting, while “I need help” expresses a request.

For the given dialogue-act classification, the dominant communicative intention is:

Request

Utterance 2

“Sure, what problem are you facing?”

The utterance asks the user for information.

Dialogue Act: Question

Utterance 3

“I do not understand machine learning.”

The user provides information about their difficulty.

Dialogue Act: Inform

Utterance 4

“I can explain the basic concepts to you.”

The agent indicates willingness to provide assistance.

Dialogue Act: Offer

4. Final Dialogue-Act Sequence

User → Request
Agent → Question
User → Inform
Agent → Offer

Sequence Representation

REQUEST
↓
QUESTION
↓
INFORM
↓
OFFER

5. Pseudocode

ALGORITHM DialogueActClassification(Conversation)

INPUT:

Conversation containing multiple utterances

OUTPUT:

Dialogue act assigned to each utterance

BEGIN

1. FOR each utterance U in Conversation DO

Preprocess U
Convert U to lowercase
Remove unnecessary symbols
Tokenize U

Extract:

Keywords

Sentence type
Question words
Intent indicators

IF U contains greeting expressions
AND greeting is the main intention THEN

Act ← Greeting

ELSE IF U contains a question word
OR U ends with a question mark THEN

Act ← Question

ELSE IF U contains request indicators
such as "need", "help", "please" THEN

Act ← Request

ELSE IF U indicates willingness
to provide assistance THEN

Act ← Offer

ELSE IF U confirms previous information THEN

Act ← Confirmation

ELSE

Act ← Inform

END IF

Store Act for U

END FOR

2. Generate the dialogue-act sequence.

3. Return all classified dialogue acts.

END

6. Applying the Algorithm

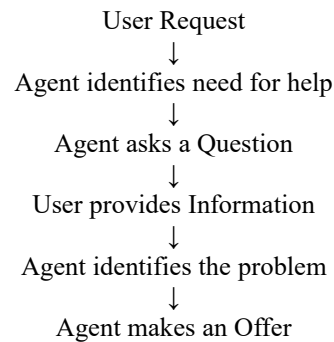
No.	Speaker	Utterance	Classified Dialogue Act	Reason
1	User	Hello, I need help with my assignment.	Request	User is asking for assistance.
2	Agent	Sure, what problem are you facing?	Question	Agent requests specific information.
3	User	I do not understand machine learning.	Inform	User provides information about the problem.
4	Agent	I can explain the basic concepts	Offer	Agent offers assistance.

No.	Speaker	Utterance	Classified Dialogue Act	Reason
		to you.		

7. How Dialogue-Act Recognition Helps a Conversational Agent

Dialogue-act recognition allows the conversational agent to understand **what the user is trying to accomplish**, rather than only identifying individual words.

For example:



This allows the agent to select an appropriate response based on the current conversational context.

Without dialogue-act recognition, the system may misunderstand a **request as information**, fail to ask a necessary question, or provide an irrelevant response.

8. Completeness and Edge Cases

A robust dialogue-act classifier should also handle cases where:

- One utterance contains more than one intention.
- A request is expressed indirectly.
- A question does not contain a question mark.
- A greeting and request occur in the same sentence.
- The meaning depends on previous conversation context.

For example:

“Hello, can you help me with my assignment?”

contains both a **Greeting** and a **Request**. The system should identify the dominant intention based on the application's dialogue policy.

Conclusion

The algorithm classifies the conversation as:

Request → Question → Inform → Offer

Dialogue-act recognition enables a conversational agent to understand user intentions and maintain an appropriate response flow. It improves **context awareness, response relevance, and conversational coherence** in multi-turn interactions.

Q5. Algorithm for Dialogue State Tracking

Given Conversation

User: “I want to book a flight.”

Agent: “Where would you like to travel?”

User: “I want to go to Delhi.”

Agent: “From which city?”

User: “From Chennai.”

The objective is to maintain the required dialogue slots and update them as information is provided by the user.

1. Problem Understanding

The conversational agent must maintain four slots:

Slot	Initial Value	Final Value
Departure City	Unknown	Chennai
Destination City	Unknown	Delhi
Travel Date	Unknown	Not provided
Number of Passengers	Unknown	Not provided

The system should update the state after every user utterance and identify which information is still missing.

2. Initial Dialogue State

Before receiving any information:

Departure City = Unknown

Destination City = Unknown

Travel Date = Unknown

Passengers = Unknown

3. State Update Process

User 1

“I want to book a flight.”

The system identifies the user's **flight-booking intent**, but no slot value is provided.

Departure City = Unknown

Destination City = Unknown

Travel Date = Unknown
Passengers = Unknown

The agent therefore asks:

“Where would you like to travel?”

User 2

“I want to go to Delhi.”

The system extracts:

Destination City = Delhi

Updated state:

Departure City = Unknown
Destination City = Delhi
Travel Date = Unknown
Passengers = Unknown

The agent then asks:

“From which city?”

User 3

“From Chennai.”

The system extracts:

Departure City = Chennai

Final state:

Departure City = Chennai
Destination City = Delhi
Travel Date = Unknown
Passengers = Unknown

4. Identify Missing Information

After processing all user responses:

Slot	Status
Departure City	Filled – Chennai
Destination City	Filled – Delhi
Travel Date	Missing
Number of Passengers	Missing

The system should therefore ask for the next missing slot, for example:

“What is your travel date?”

After receiving the travel date, it can ask for the number of passengers.

5. Pseudocode

ALGORITHM DialogueStateTracking(Conversation)

INPUT:

Multi-turn flight booking conversation

OUTPUT:

Updated dialogue state and next appropriate question

BEGIN

1. Initialize dialogue state:

DepartureCity $\leftarrow$ Unknown

DestinationCity $\leftarrow$ Unknown

TravelDate $\leftarrow$ Unknown

Passengers $\leftarrow$ Unknown

2. FOR each user utterance U DO

Preprocess U

Extract entities and slot values from U

IF U contains a destination city THEN

DestinationCity $\leftarrow$ ExtractDestination(U)

END IF

IF U contains a departure city THEN

DepartureCity $\leftarrow$ ExtractDeparture(U)

END IF

IF U contains a travel date THEN

TravelDate $\leftarrow$ ExtractDate(U)

END IF

IF U contains number of passengers THEN

Passengers $\leftarrow$ ExtractPassengerCount(U)

END IF

END FOR

3. Check all required slots.

IF DepartureCity = Unknown THEN

NextQuestion $\leftarrow$ "Where are you travelling from?"

ELSE IF DestinationCity = Unknown THEN

NextQuestion $\leftarrow$ "Where would you like to travel?"

ELSE IF TravelDate = Unknown THEN

NextQuestion $\leftarrow$ "What is your travel date?"

ELSE IF Passengers = Unknown THEN

NextQuestion $\leftarrow$ "How many passengers are travelling?"

```
ELSE
  NextQuestion ← "All required information has been collected."
```

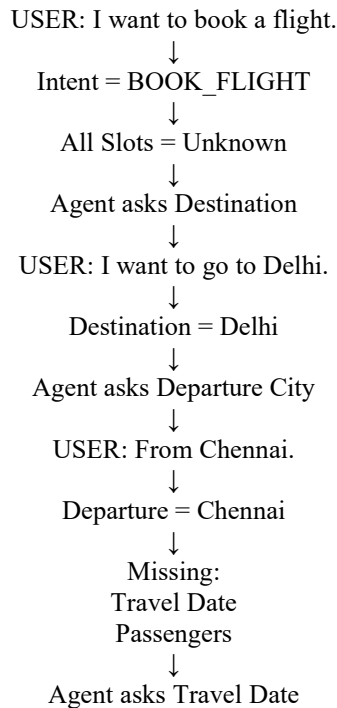
```
END IF
```

```
4. Return:
  Dialogue State
  NextQuestion
```

```
END
```

6. Dialogue State Representation

The conversation can be represented as:



7. Final Dialogue State

Intent = Book Flight
Departure City = Chennai
Destination City = Delhi
Travel Date = Unknown
Passengers = Unknown
Next Appropriate Question

“What is your travel date?”

8. How Dialogue State Tracking Improves Coherence

Dialogue State Tracking maintains information from previous turns so that the conversational agent does not repeatedly ask for information that the user has already provided.

For example, after the user says:

“From Chennai.”

the system remembers:

Departure City = Chennai

Therefore, it does not ask “**Which city are you departing from?**” again. Instead, it moves to the next missing slot, **Travel Date**.

This improves:

- **Context awareness**
- **Conversation continuity**
- **Response relevance**
- **User experience**
- **Multi-turn interaction accuracy**

Conclusion

The proposed dialogue state tracking algorithm successfully maintains the flight-booking information across multiple turns. After processing the given conversation, the system correctly identifies **Chennai as the departure city** and **Delhi as the destination**, while detecting that **travel date and number of passengers are still missing**. Thus, the agent can generate the next appropriate question without losing the conversation context.